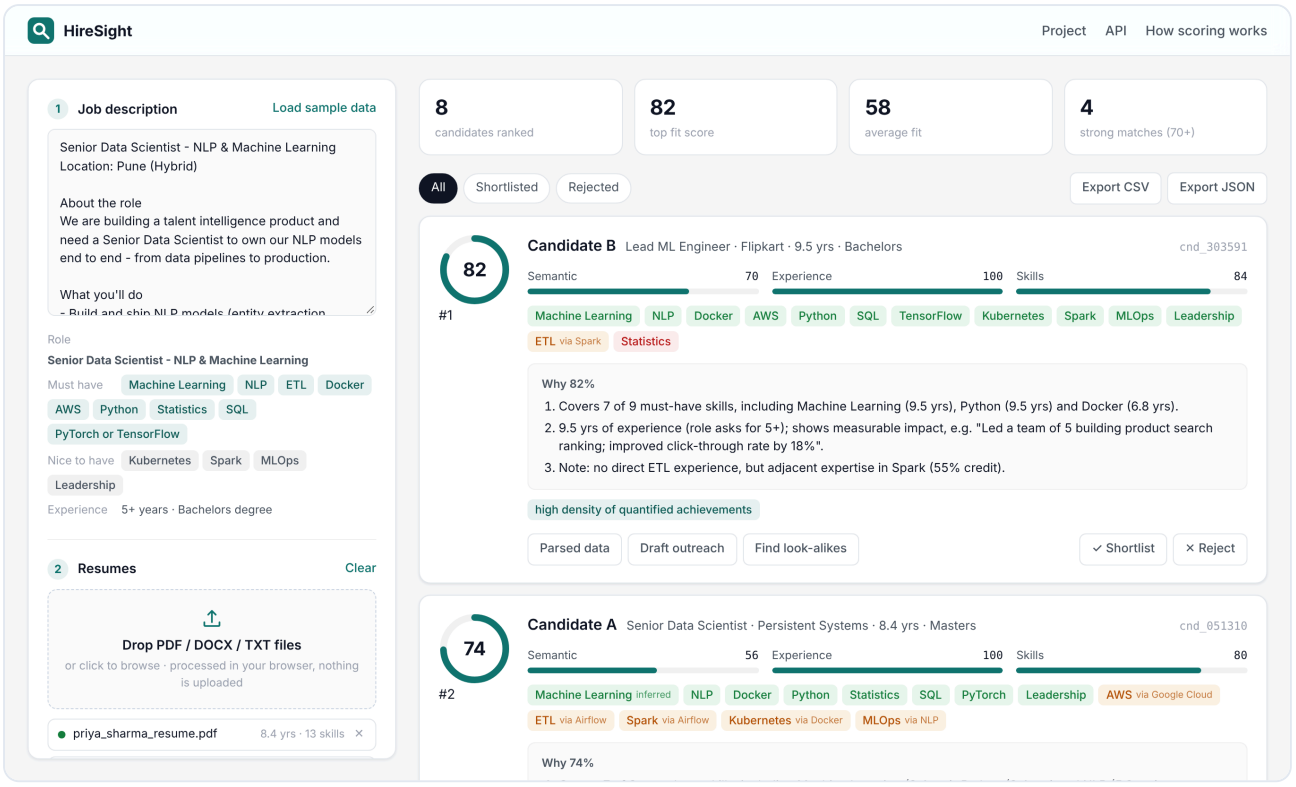
INTERNSHIP PROJECT REPORT · DATA SCIENCE

Resume Parsing and Candidate Ranking for Recruiters

HireSight: a layout-aware resume parser and explainable candidate ranking engine



Submitted by	**Omkar Vijay Ghate**
College	Bajaj Degree College
Organization	Persevex
Domain	Data Science
Year	2026
LinkedIn	linkedin.com/in/-omkar-ghate-profile

Abstract

Recruiters spend a large share of their day screening resumes, and most applicant tracking systems still match keywords literally: a candidate who writes "GCP" is invisible to a search for "Google Cloud". This project builds HireSight, a system that parses PDF and DOCX resumes into structured JSON and ranks candidates against a job description (JD). Parsing is layout-aware, so two-column resumes are read column by column. Skills are normalised through a hand-built ontology of 72 canonical skills with aliases, implied skills and adjacency scores. Ranking runs in three stages: TF-IDF retrieval, a passage-level rerank, and a weighted fit score (50% semantic, 30% experience, 20% skills). Every score comes with a three-point explanation, and a blind mode removes personal and identity information before scoring. On a sample pool of eight resumes the system put the four relevant senior profiles on top and kept a keyword-stuffed fresher resume in the middle of the table. It processes 200 resumes in about 305 ms.

Contents

1. Introduction and problem statement
2. Objectives
3. System architecture
4. Parsing engine
5. Skill ontology
6. Ranking algorithm
7. Ethical AI and bias mitigation
8. Implementation
9. Results and evaluation
10. Limitations and future scope
11. Conclusion

1. Introduction and Problem Statement

A single job posting can attract hundreds of applicants. Faced with that volume, recruiters skim each resume for a few seconds, and the outcome depends heavily on fatigue and on how a resume happens to be worded. Traditional ATS software does not help much, for three reasons that the project brief calls out:

- **The two-column problem.** Modern resumes put skills or contact details in a sidebar. Naive text extraction reads straight across the page and mixes the sidebar into the work history.
- **Keyword games.** Exact-match search rewards candidates who paste every JD keyword into their resume (sometimes in invisible white text), and penalises honest candidates who describe the same skill in other words.
- **No explanation.** A recruiter who gets a score has to defend it to a hiring manager. A bare number is not enough.

2. Objectives

1. **Multi-format, layout-aware parsing** of PDF and DOCX resumes that keeps sections intact.
2. **Granular extraction**: not just skills, but how long and in what role they were used, separating a five-year job from a three-month bootcamp.
3. **A semantic ranking engine** that rewards meaning and adjacent skills rather than exact words.
4. **An explainable recruiter UI** with reasons for every rank, plus blind screening.

3. System Architecture

The whole engine is written as small JavaScript ES modules in `lib/`. The same modules run in the browser (the recruiter app) and inside Vercel serverless functions (the REST API), so there is one implementation of every rule. File extraction runs in the browser, which means resumes never leave the recruiter's machine when the web app is used.

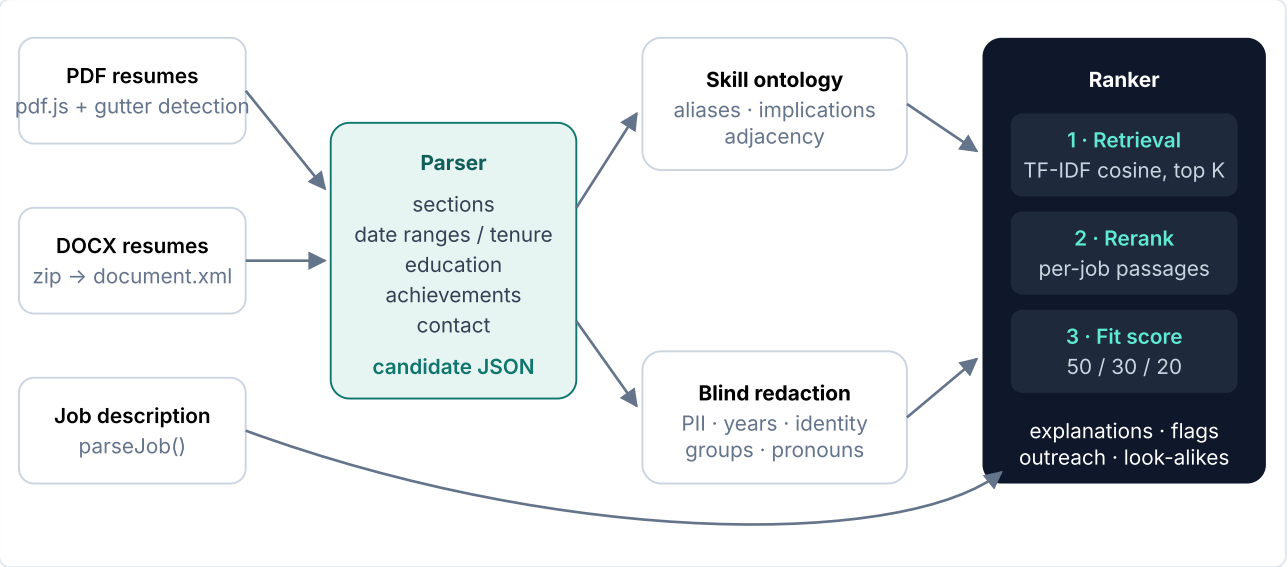


Figure 1. Data flow from files to ranked, explained results.

MODULE	RESPONSIBILITY
app/extract.js	PDF text with coordinates (pdf.js), gutter detection, hidden-text filter; DOCX zip reader
lib/parser.js	Sections, jobs and durations, education, contact, achievements, JD requirements
lib/ontology.js	Canonical skills, aliases, implications, family adjacency
lib/redact.js	Blind-mode redaction with a log of what was removed
lib/ranker.js	Vectorising, retrieval, rerank, fit score, explanations, look-alikes, outreach
api/*.js	/api/health, /api/parse, /api/rank on Vercel

4. Parsing Engine

4.1 Spatial-aware PDF extraction

pdf.js returns every text run with its transform matrix, which gives its x/y position and font size. For each page, the extractor tries vertical lines between 25% and 75% of the page width and counts how many text runs cross each one. The line with the fewest crossings is accepted as a column gutter only if both sides hold at least 15% of the text and almost nothing crosses it. Runs that span the gutter (usually a centred name banner) are read first, then the left column, then the right. Inside a column, runs are grouped into lines by y position; a large vertical gap or a bigger font starts a new block, which helps the section splitter. Text smaller than 3.5 pt is discarded, since it is almost always hidden keyword stuffing.

4.2 DOCX

A .docx file is a zip archive. Instead of adding a library, the extractor reads the zip central directory, inflates `word/document.xml` with the browser's built-in `DecompressionStream('deflate-raw')`, and walks the paragraphs. List paragraphs become bullet lines.

4.3 Sections and entities

- **Sections** are found with header patterns (Experience, Work History, Internships, Education, Skills, Projects…). Wrapped bullet lines from PDFs are joined back together first.
- **Jobs**: any line with a date range (`Jan 2019 - Present`, `03/2020 – 06/2022`, `2018 - 2020`) starts a job. Title and company are split from the same or previous line using role words (engineer, analyst, intern…) and the skill dictionary, so "Applied Scientist – NLP | Microsoft" gives company = Microsoft, not NLP.
- **Experience** is summed month by month so overlapping jobs count once. Internships, bootcamps and short course projects count at half weight.
- **Per-skill tenure**: months of every job that mentions a skill (or a skill implying it).
- **Achievements**: bullets that start with an action verb and contain a metric (%, x, ₹/$, "12M users", "5 teams"). The ratio of quantified bullets is used as a quality signal.
- **Education**: highest degree level, major, graduation year and CGPA/percentage. Abbreviations like "BE" or "MS" are matched case-sensitively so "must be" in a JD is not read as a degree.

4.4 Output

Every resume becomes a JSON object. An excerpt for one sample candidate:


```
{
  "candidate_id": "cnd_051310",
  "contact": {
    "name": "Priya Sharma",
    "email": "priya.sharma.ds@gmail.com",
    "linkedin": "linkedin.com/in/priya-sharma-ds"
  },
  "total_years_experience": 8.4,
  "experience": [
    {
      "title": "Senior Data Scientist",
      "company": "Persistent Systems",
      "start": "2021-01",
      "end": "Present",
      "duration_months": 71,
      "junior": false,
      "technologies_used": [
        "NLP",
        "Google Cloud",
        "Leadership",
        "Transformers"
      ]
    }
  ],
  "education": {
    "degree_level": "Masters",
```

```
    "major": "Computer Science",
    "graduation_year": 2018
  },
  "skills": {
    "explicit": [
      "NLP",
      "Statistics",
      "Google Cloud",
      "scikit-learn",
      "Transformers",
      "Leadership",
      "…"
    ],
    "implied": [
      "Deep Learning",
      "Machine Learning"
    ],
    "tenure_years": {
      "NLP": 5.9,
      "Google Cloud": 5.9,
      "Leadership": 5.9,
      "Transformers": 5.9
    }
  },
  "achievements": [
    {
      "verb": "built",
      "metric": "71"
    },
    {
      "verb": "designed",
      "metric": "2M"
    }
  ],
  "quantified_ratio": 0.71
}
```


5. Skill Ontology

Raw strings are not enough for ranking, so every skill mention is mapped to a canonical node. The ontology has 72 nodes across languages, frameworks, databases, cloud, DevOps, data engineering, ML/NLP, BI, mobile and soft skills.

- **Normalisation**: "K8s", "AWS EKS", "helm" and "Kubernetes" all map to *Kubernetes*; "GCP", "BigQuery", "GKE" map to *Google Cloud*. Short ambiguous tokens (Go, R, Excel, Spark) only count when written like a skill, so "ready to go the extra mile" does not add Go.
- **Implication**: PyTorch implies Python and Deep Learning, which implies Machine Learning. Inferred skills get 0.85 credit.
- **Adjacency**: skills in the same family transfer partially: React ↔ Vue.js 0.7, AWS ↔ Azure ↔ Google Cloud 0.7, PyTorch ↔ TensorFlow 0.75. Related families get a smaller cross-credit (e.g. SQL databases ↔ NoSQL 0.4).

6. Ranking Algorithm

6.1 Stage 1: retrieval

The JD and all (redacted) resumes are tokenised. Each recognised skill becomes one canonical token such as `skill:google_cloud`, weighted 1.6× in the vector; remaining words are lower-cased, stop-worded and lightly stemmed. TF-IDF vectors with sub-linear term frequency are compared with cosine similarity:

```
cos(J, R) = (J · R) / (‖J‖ · ‖R‖)
```

The top K candidates (default 100) continue to stage 2. The brief suggests a Sentence-Transformer bi-encoder here; TF-IDF was chosen because it runs instantly in a browser and on a free serverless tier, and the canonical skill tokens remove the biggest weakness of word matching. The vectoriser is isolated so real embeddings can replace it later.

6.2 Stage 2: passage rerank

A cross-encoder would read the JD and resume together. As a lightweight stand-in, the JD is compared with each job block separately. The most recent job has weight 1.0 and each older one 0.12 less (minimum 0.6). The rerank score blends the whole-document and passage views and is then mapped to 0–1:

```
s = 0.5·retrieval + 0.3·best_passage + 0.2·avg_passage
semantic = s / (s + 0.18)
```

6.3 Stage 3: fit score

```
fit = 100 × (0.5·semantic + 0.3·experience + 0.2·skills)
```

- **Experience** = 1 if years ≥ required, otherwise `exp(−2 · shortfall / required)`. One year short of five keeps 67%, three years short keeps 30%.
- **Skills**: for each JD requirement the best credit is taken: 1.0 if used in a job/project, 0.85 if inferred, 0.75 if it appears *only* in the skills list, or the adjacency value of the closest skill. Must-haves weigh 1, nice-to-haves 0.5, soft skills half. "PyTorch or TensorFlow" is one requirement satisfied by either.
- **Why not plain Jaccard?** Jaccard divides by the union of both skill sets, so a candidate with a broad, honest skill list scores lower for knowing more. Jaccard is still computed and shown in the details view, but the score uses the JD-anchored coverage above.

6.4 Explanations

Each candidate gets three bullets generated from the breakdown: (1) which must-have skills are covered and for how long; (2) total experience against the requirement plus the strongest quantified achievement; (3) the main caveat — keyword-stuffing flag, adjacent skill used instead of a required one, missing skills, a per-skill year gap, or an experience shortfall. Example: *"Note: no direct AWS experience, but adjacent expertise in Google Cloud (70% credit)."* The bullets are

template-based rather than LLM-generated, so they are deterministic, free, and can only state facts that exist in the parsed data.

6.5 Recruiter features

- **Look-alike search**: nearest neighbours of a chosen candidate in the same vector space.
- **Outreach drafts**: a first email quoting the candidate's top achievement and the matched skills.
- **Feedback loop**: shortlist/reject decisions are compared with the scores. When recruiters disagree with the model on several candidates, the app shows which component separates their choices and suggests a weight change.

7. Ethical AI and Bias Mitigation

Before anything is scored, a redaction step removes information that should not affect a hiring decision. The ranker only ever sees the redacted text; dates for experience are taken from the original parse.

REMOVED / GENERALISED	REASON
Name, email, phone, profile links, address lines	Identity; names can signal gender, religion, region or caste
Graduation years on education lines	Age
Date of birth, gender, religion, caste, marital status, nationality, father's name lines	Common on Indian resumes; protected attributes
Identity-linked groups ("Muslim Women's Students Association")	Replaced by "University Association" so the leadership role is kept
he / she / his / her, Mr / Ms	Gender

The app's blind mode also hides names in the results list and labels people as Candidate A, B, C. Embedding-level debiasing (projecting vectors away from a gender direction) is listed in the brief; it applies to neural embeddings and is noted under future scope, since TF-IDF on redacted text has no learned gender direction to remove.

8. Implementation

LAYER	CHOICE
Frontend	Plain HTML, CSS and JavaScript modules; no framework or build step
PDF	pdf.js 6 (vendored, served from the same origin)
Backend	Vercel serverless functions (Node 18+) reusing `lib/`
Storage	None server-side; shortlist decisions in the browser's localStorage
Testing	node:test — 18 tests across parser, ontology, redaction, ranker and API handlers
Hosting	Vercel (static + functions)

REST API

ENDPOINT	INPUT	OUTPUT
GET /api/health	–	status, ontology size
POST /api/parse	`{text}` or `{job}`	candidate JSON + redaction summary, or parsed JD
POST /api/rank	`{job, resumes[], weights?, blind?}`	ranked results with breakdown, flags, explanations

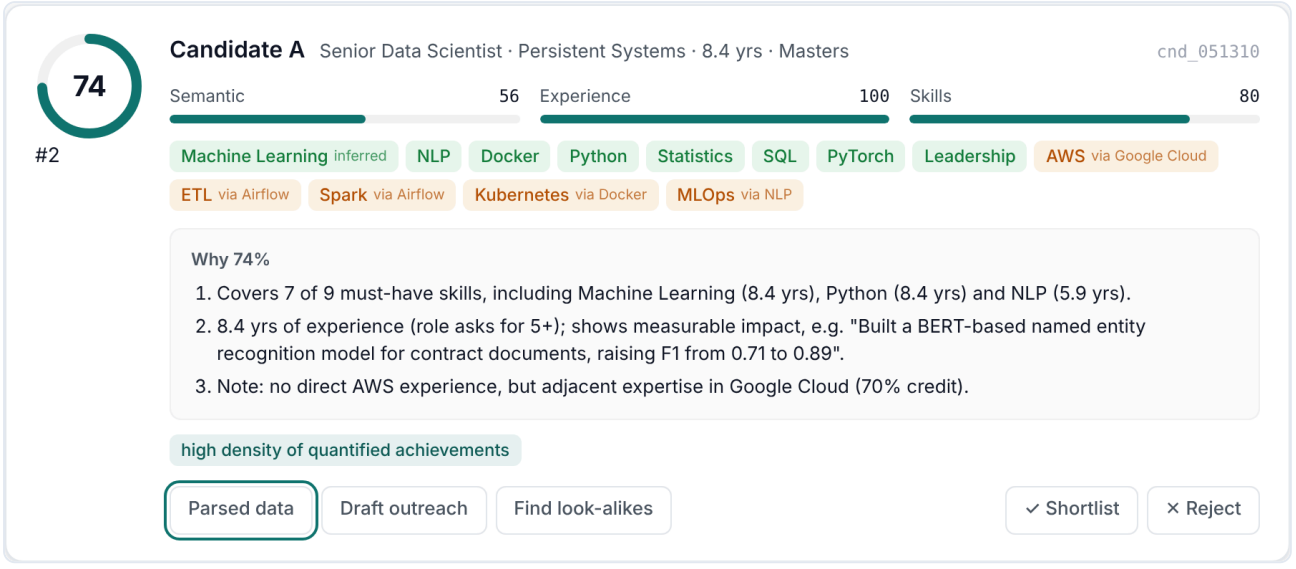


Figure 2. A ranked candidate card: fit score, component bars, matched (green), adjacent (amber) and missing (red) skills, and the three-point explanation.

9. Results and Evaluation

The sample pool contains eight fictional resumes written to cover typical cases: strong matches on different clouds, an NLP researcher, a data analyst, a backend engineer, a frontend developer, and a fresher whose skills section lists 22 technologies. The JD is a Senior Data Scientist (NLP) role requiring 5+ years. Detected must-haves: Machine Learning, NLP, ETL, Docker, AWS, Python, Statistics, SQL, PyTorch / TensorFlow.

#	CANDIDATE	FIT	SEM	EXP	SKILLS	JACC.	MAIN CAVEAT
1	**Rahul Verma** Lead ML Engineer · 9.5 yrs	**81.8**	0.70	1.00	0.84	0.67	Note: no direct ETL experience, but adjacent expertise in Spark (55% credit).
2	**Priya Sharma** Senior Data Scientist · 8.4 yrs	**74.3**	0.56	1.00	0.80	0.38	Note: no direct AWS experience, but adjacent expertise in Google Cloud (70% credit).
3	**Arjun Nair** Machine Learning Engineer · 7.5 yrs	**73.6**	0.55	1.00	0.80	0.37	Note: no direct PyTorch / TensorFlow experience, but adjacent expertise in scikit-learn (75% credit).
4	**Fatima Sheikh** Applied Scientist · 6.4 yrs	**73.5**	0.56	1.00	0.77	0.44	Note: no direct AWS experience, but adjacent expertise in Azure (70% credit).
5	**Karan Mehta** Data Science Intern · 0.3 yrs	**45.2**	0.51	0.15	0.76	0.46	Note: 19 skills are listed but never used in any job or project (possible keyword stuffing).
6	**Sneha Patil** Senior Software Engineer · 9.4 yrs	**44.6**	0.17	1.00	0.31	0.08	Note: no direct AWS experience, but adjacent expertise in Azure (70% credit).
7	**Aisha Khan** Data Analyst · 4.3 yrs	**43.4**	0.28	0.76	0.33	0.17	Note: no direct ETL experience, but adjacent expertise in SQL (35% credit).
8	**Vikram Singh** Frontend Developer · 5.1 yrs	**30.5**	0.00	1.00	0.02	0.00	Note: missing Machine Learning, NLP and ETL.

Observations

- The four candidates with real ML/NLP work and 6+ years take the top four places; unrelated profiles (Java backend, frontend) are at the bottom.
- The keyword-stuffed fresher matches almost every must-have skill on paper, but the skills were never used in any job, the experience is two 3-month stints at half weight, and the stuffing flag fires. Rank: 5th of 8.
- Candidates on Google Cloud or Azure are not rejected for an AWS requirement; they get 70% credit and a clear note.
- Plain Jaccard would rank the fresher (0.46) above three of the four strong candidates, which is why it is not used as the skill score.

Parsing checks

- Two-column PDF (sidebar with skills/education): gutter found, columns read in order, both jobs and all bullets recovered; a 1 pt white "AWS Kubernetes Spark..." line was dropped.
- DOCX with heading styles and bullet lists: all sections, both jobs and the 9.5-year total recovered.
- Unit tests cover date formats, overlapping jobs, internship weighting, alias collapse, ambiguous tokens, degree abbreviations and every redaction category.

18 / 18	190 ms	115 ms	0
automated tests passing	to parse 200 resumes (Node)	to rank 200 resumes	resume files uploaded to a server

10. Limitations and Future Scope

Current limitations

- Scanned PDFs without a text layer are not supported (no OCR in the browser build).
- The ontology is hand-written and tech-focused; non-technical roles rely mostly on the semantic score.
- Rule-based parsing misses unusual date formats and very creative layouts (three columns, text in images).
- Sample size: evaluation used a small, hand-built pool; a proper study needs labelled recruiter decisions.

Future work

- Replace stage 1 with sentence embeddings in a vector database, and stage 2 with a cross-encoder, keeping the same interfaces.
- Fine-tune a token-classification model (e.g. RoBERTa with BIO tags) on labelled resumes for entities the rules miss.
- Tesseract OCR for scanned resumes, served from a worker.
- Use logged shortlist/reject decisions to learn the 50/30/20 weights per role instead of suggesting them.
- Flight-risk estimation from average tenure, and salary-band suggestions from market data, as listed in the brief.

11. Conclusion

HireSight shows that a recruiter-facing ranking tool can be accurate enough to be useful without being a black box. Reading resumes the way they are laid out, normalising skills through an ontology, and combining semantic similarity with explicit experience and skill rules produced a ranking that matched intuition on the test pool, resisted keyword stuffing, and gave partial credit where a human recruiter would. Every number is explained, personal information is removed before scoring, and the full pipeline runs in the browser in well under a second for hundreds of resumes.